Heart Disease Risk Prediction
End-to-End MLOps Pipeline

Machine Learning Operations (MLOps) AIMLCZG523 - Assignment 01

Student: **[your name / ID]** | Repository: https://github.com//heart-disease-mlops

1. Introduction

This project builds a classifier that predicts the presence of heart disease from 13 routine clinical measurements (UCI Heart Disease, Cleveland subset) and delivers it the way a production team would: reproducible data and training code, tracked experiments, automated tests and CI, a Docker image, Kubernetes manifests and Prometheus/Grafana monitoring. The emphasis is on the engineering around the model rather than on squeezing the last point of accuracy out of 302 patients.

Key results. The selected model is a logistic-regression pipeline with a hold-out ROC-AUC of **0.918** (accuracy 0.836, precision 0.846, recall 0.786) and a nested cross-validated ROC-AUC of 0.907 +/- 0.039. The service passes 38 automated tests (78% line coverage), reproduces from a fresh virtual environment using only `requirements-dev.txt`, and exposes `/predict` with prediction and confidence.

Honest scope note. The model is for education, not clinical use; the test set has only 61 patients so reported test metrics have wide uncertainty (roughly +/- 0.1 on accuracy).

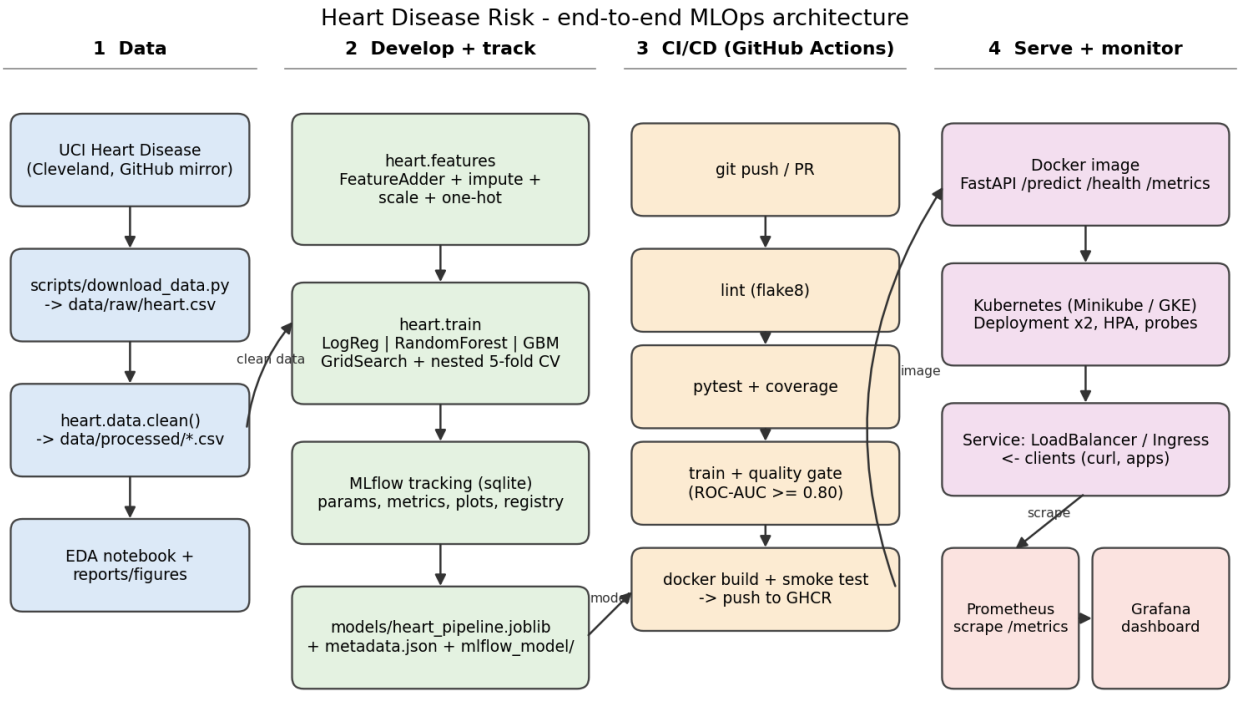


Figure 1 - End-to-end architecture: data, development/tracking, CI/CD and serving/monitoring.

2. Setup and installation

```
git clone https://github.com/<your-username>/heart-disease-mlops.git && cd heart-disease-mlops
python3 -m venv .venv && source .venv/bin/activate        # Python 3.12
pip install -r requirements-dev.txt
export PYTHONPATH=src
python scripts/download_data.py      # data/raw/heart.csv (copy committed)
python -m heart.data                 # cleaned dataset -> data/processed/
python -m heart.train                # tune, evaluate, log to MLflow, save models/
pytest --cov=src/heart && flake8 src tests scripts
uvicorn heart.api:app --port 8000    # or: docker build -t heart-disease-api:1.0.0 .
```

Dependencies are pinned in `requirements.txt` (training) and `requirements-serve.txt` (slim inference image without MLflow/plotting libraries). To verify reproducibility, a pristine copy of the repository without any generated artefacts was installed into a new virtualenv and the whole flow (download, clean, train, test) was executed successfully (Figure 2).

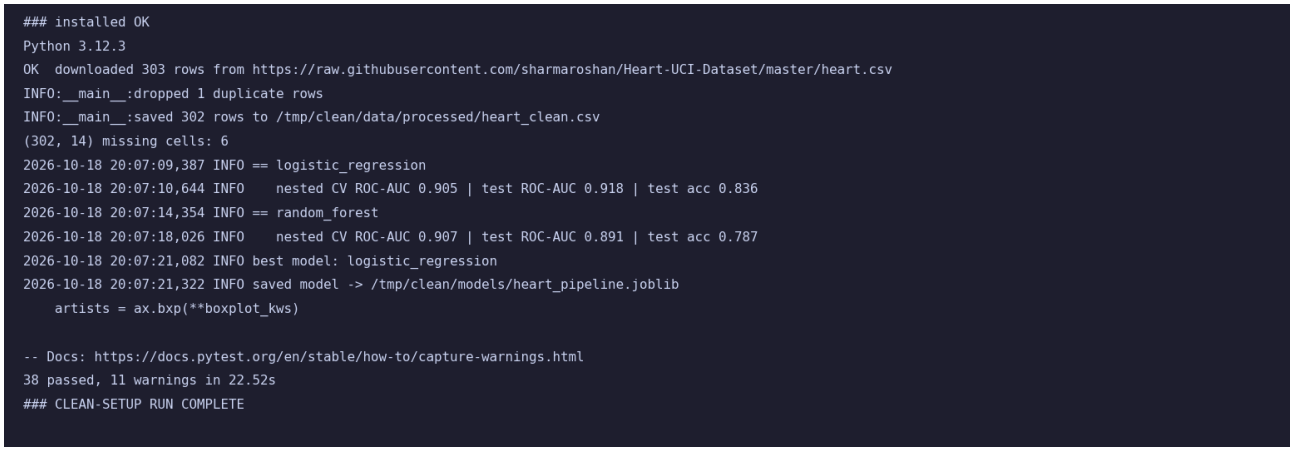


Figure 2 - Terminal output of the clean-environment run (real output; text rendered as an image).

3. Data acquisition, cleaning and EDA

Source. The UCI Machine Learning Repository host was not reachable from the build environment, so `scripts/download_data.py` downloads the widely mirrored 14-column CSV of the same Cleveland data from GitHub (with a second mirror as fallback and content validation). The README explains the official UCI route and its different coding.

Cleaning (`heart.data.clean`): (i) schema validation; (ii) numeric coercion; (iii) invalid category codes (`ca = 4`, `thal = 0`) become missing; (iv) one exact duplicate removed; (v) **target inversion**. In this mirror `target = 1` encodes *absence* of disease: those patients have a higher maximum heart rate and less exercise angina, and the class counts match UCI's 164 healthy / 139 diseased. After cleaning, `1 = disease present`. Result: 302 patients, 164 negative / 138 positive, and 6 missing cells (4 in ca, 2 in thal) - exactly UCI's documented missing values. Missing values are imputed inside the model pipeline so that statistics come only from training folds.

Feature group	Columns	Treatment
Continuous	age, trestbps, chol, thalach, oldpeak	median imputation, standard scaling
Engineered	pct_max_hr = thalach/(220-age); st_depression_with_angina = oldpeak x exang	as continuous
Categorical	sex, cp, fbs, restecg, exang, slope, ca, thal	most-frequent imputation, one-hot (unknown ignored)

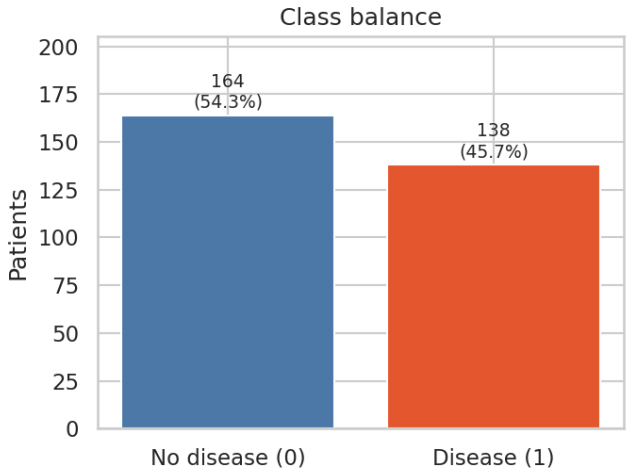


Figure 3 - Class balance (54% / 46%): near-balanced, so accuracy is meaningful; recall and ROC-AUC are reported too.

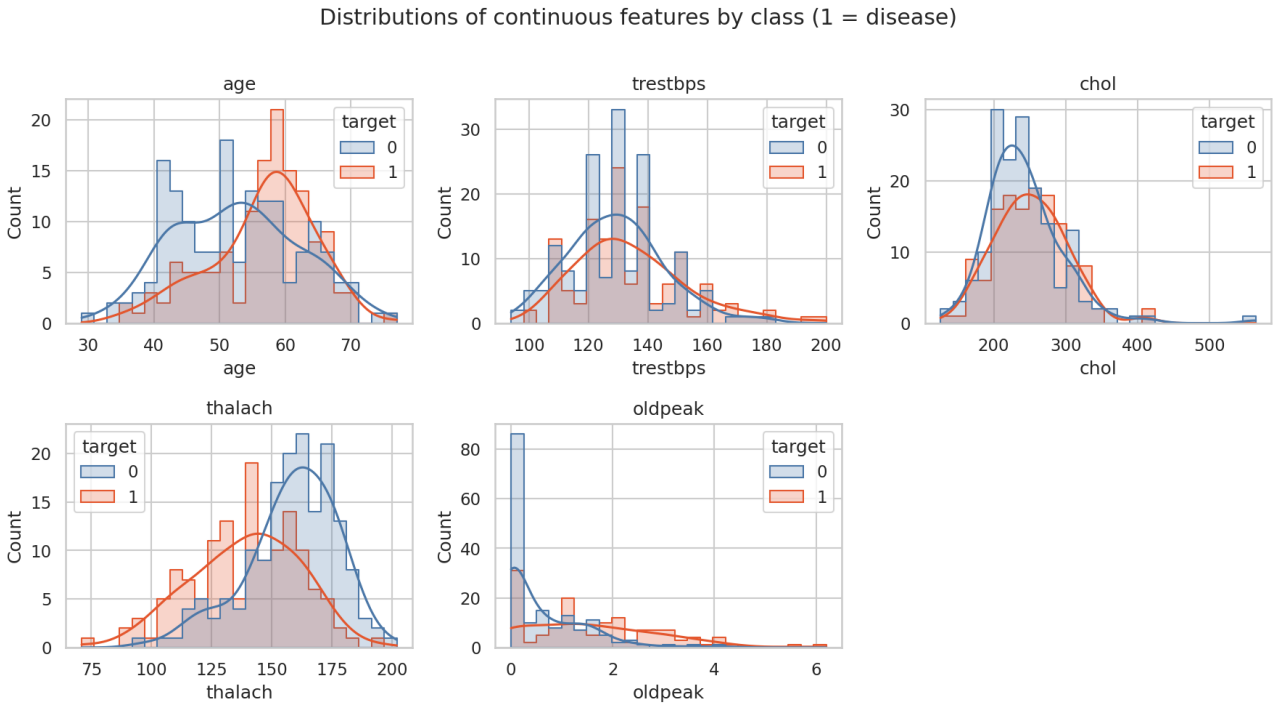


Figure 4 - Continuous features by class. Disease cases are older, reach a lower maximum heart rate and show larger ST depression.

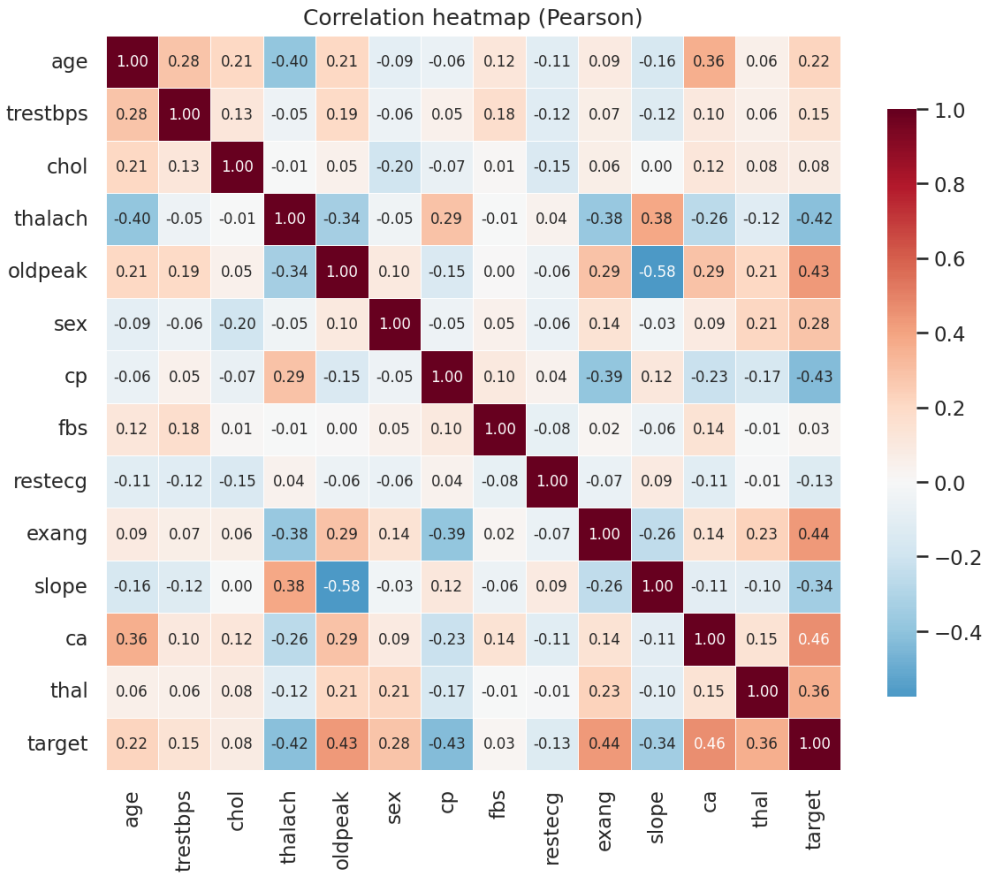


Figure 5 - Correlation heatmap. Strongest links to the target: ca (+0.46), exang (+0.44), cp (-0.43), oldpeak (+0.43), thalach (-0.42); no predictor pair above |r| = 0.58, so no feature was dropped.

Other observations. `cp = 0` covers 47% of patients and 73% of them have disease (18-30% for other codes); exercise angina raises the disease rate from 31% to 77%. Cholesterol above 400 mg/dl and oldpeak above 5 are rare but plausible, so they were kept (scaling and the chosen models tolerate them).

4. Feature engineering and model development

All preprocessing and the estimator live in a single scikit-learn `Pipeline`, guaranteeing identical transformations in training, cross-validation, notebooks, the API and the container. Data were split 80/20 (stratified, seed 42): 241 training and 61 test patients. Three classifiers were compared, each tuned with grid search (5-fold stratified, ROC-AUC):

Model	Hyper-parameter grid
Logistic regression	C in {0.01, 0.1, 0.5, 1, 5, 10}; class_weight in {None, balanced} (12 combinations)
Random forest	n_estimators {100, 300}; max_depth {3, 5, None}; min_samples_leaf {1, 3, 5} (18)
Gradient boosting	n_estimators {50, 100, 200}; learning_rate {0.03, 0.1}; max_depth {2, 3} (12)

Evaluation protocol. To avoid optimistic estimates from tuning on the same folds, performance was estimated with *nested* cross-validation (3-fold inner search inside a 5-fold outer loop) on the training split using accuracy, precision, recall, F1 and ROC-AUC; each tuned model was then scored once on the hold-out test set.

Model	CV ROC-AUC	CV accuracy	CV recall	Test ROC-AUC	Test acc.	Test recall
Logistic regression (selected)	0.907 +/- 0.039	0.846	0.809	0.918	0.836	0.786
Random forest	0.910 +/- 0.036	0.830	0.791	0.894	0.787	0.679
Gradient boosting	0.900 +/- 0.030	0.830	0.809	0.879	0.770	0.643

Model selection. The three CV ROC-AUC values differ by 0.01, well within the fold-to-fold standard deviation (0.03-0.04), so choosing the nominal winner (random forest, 0.910) would mean choosing noise. I therefore

applied the **one-standard-error rule**: among models within one SD of the best CV score, take the simplest. This rule uses CV results only - the test set played no part in the decision. Logistic regression is selected; it is also the most interpretable and had the best CV accuracy and F1. Best parameters and all metrics are stored in `models/metadata.json`.

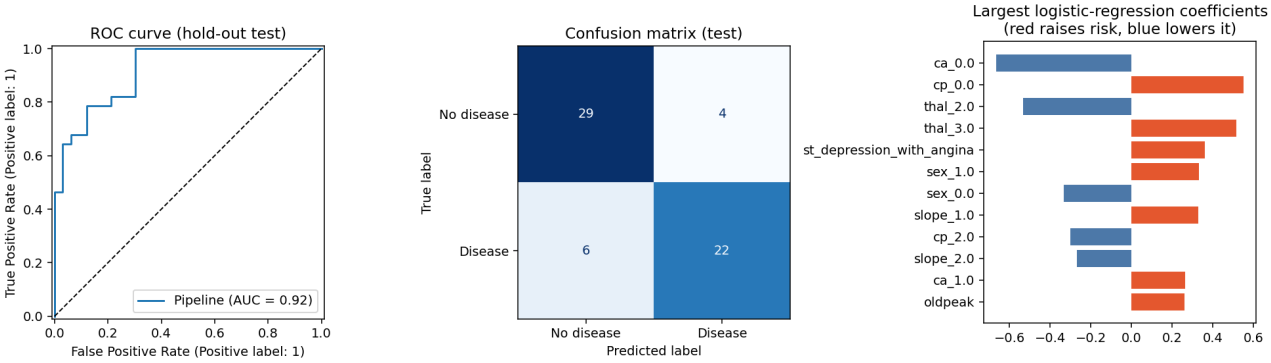


Figure 6 - Selected model on the hold-out test set: ROC curve, confusion matrix and the largest coefficients.

Limitations. With 61 test patients, one extra error moves accuracy by 1.6 points. The default 0.5 threshold gives recall 0.79; for screening a lower threshold could be configured (notebook 02 shows the precision/recall trade-off).

```
$ python -m heart.train      # nested 5-fold CV on train split (mean +/- sd) and hold-out test
                 model      cv_accuracy     cv_precision       cv_recall           cv_f1      cv_roc_auc  test_accuracy  test_precision  test_recall  test_f1  test_roc_auc
logistic_regression 0.846 +/- 0.032 0.858 +/- 0.071 0.809 +/- 0.088 0.827 +/- 0.038 0.907 +/- 0.039          0.836           0.846        0.786    0.815         0.918
      random_forest 0.830 +/- 0.040 0.845 +/- 0.089 0.791 +/- 0.102 0.808 +/- 0.045 0.910 +/- 0.036          0.787           0.826        0.679    0.745         0.894
  gradient_boosting 0.830 +/- 0.025 0.830 +/- 0.086 0.809 +/- 0.088 0.812 +/- 0.028 0.900 +/- 0.030          0.770           0.818        0.643    0.720         0.879
selected: logistic_regression (one-standard-error rule on nested-CV ROC-AUC)
```

Figure 7 - Output of python -m heart.train (nested CV and test metrics for all candidates).

5. Experiment tracking (MLflow)

`heart.train` opens one parent run and one child run per model in the experiment *heart-disease-risk* (SQLite backend `mlflow.db`). Each child run logs: parameters (model, best hyper-parameters, seed, split sizes, CV folds, data MD5, library versions); metrics (tuning score, nested-CV mean and SD for five metrics, five test metrics); artifacts (ROC curve, confusion matrix, feature importance, full grid-search results CSV, classification report) and the fitted pipeline with a signature and input example. The parent run stores the comparison table and chart, the selection rule and the selected model, which is registered as *heart-disease-classifier* in the MLflow model registry.

```
$ python -c "mlflow.search_runs(...)"   # experiment: heart-disease-risk
                    run  cv_roc_auc_mean  cv_accuracy_mean  cv_recall_mean  test_roc_auc  test_accuracy  test_recall
training-20261018-195635                -                 -               -             -              -            -
     logistic_regression            0.907             0.846           0.809         0.918          0.836        0.786
           random_forest            0.910             0.830           0.791         0.894          0.787        0.679
       gradient_boosting            0.900             0.830           0.809         0.879          0.770        0.643

registered model versions: [('heart-disease-classifier', 1)]
```

Figure 8 - MLflow runs queried via the Python API (real output).

SCREENSHOT TO ADD: Figure 9 - MLflow UI: experiment table comparing the three runs.
save as screenshots/15_mlflow_ui_runs.png, then re-run docs/build_report.py

SCREENSHOT TO ADD: Figure 10 - MLflow UI: parameters, metrics and artifacts of one run.
save as screenshots/16_mlflow_ui_run_detail.png, then re-run docs/build_report.py

6. Model packaging and reproducibility

- **Formats:** `models/heart_pipeline.joblib` (used by the API) and an MLflow model directory `models/mlflow_model/` (MLmodel, conda.yaml, requirements.txt). The serialized object is the *whole* pipeline (feature engineering, imputers, scaler, encoder, classifier), so no preprocessing code is duplicated at serving time.
- **Environment:** pinned `requirements*.txt`; Python 3.12; fixed random seeds; training verified deterministic by a unit test.
- **Lineage:** `metadata.json` records model version, training time, metrics, feature list, library versions and the MD5 of the training data.
- **Clean-setup proof:** Figure 2 (fresh venv, requirements only, 38 tests passing).

7. CI/CD pipeline and automated testing

Tests (pytest, 38 tests, 78% coverage): data cleaning (invalid codes, duplicates, target inversion, published-statistics check), feature engineering (values, immutability, finite output with missing data, unseen categories, standardisation), model training (metric ranges, determinism, artifact round-trip, optional-field handling, risk monotonicity, selection rule), the download script (mocked network and mirror fallback), EDA output and the API (valid/invalid inputs, metrics endpoint, request-ID correlation, no patient payload in logs).

```
$ pytest --cov=src/heart -q
.....................................                                    [100%]
Name                    Stmts   Miss  Cover
-----------------------------------------
src/heart/__init__.py       1      0   100%
src/heart/api.py          120      7    94%
src/heart/config.py        19      0   100%
src/heart/data.py          38      3    92%
src/heart/eda.py           84      2    98%
src/heart/features.py      28      1    96%
src/heart/predict.py       20      0   100%
src/heart/train.py        211    103    51%
-----------------------------------------
TOTAL                     521    116    78%
38 passed, 11 warnings in 41.83s
```

Figure 11 - pytest with coverage (real output).

Workflow (`.github/workflows/ci.yml`): *lint* (flake8) → *test* (pytest, coverage, JUnit XML uploaded as artifact) → *train* (download, clean, train with MLflow, quality gate: hold-out ROC-AUC must be at least 0.80; model, `mlflow.db` and training log uploaded) → *docker* (build the image, run it, wait for /health, POST a patient to /predict, check /metrics; container logs uploaded) → *publish* (push image to GHCR on main only). Jobs depend on each other with `needs`, so any non-zero exit stops the pipeline and the step log shows the failure. To demonstrate this, a deliberate bug was introduced into the feature code in a throw-away copy: the test step failed and returned exit code 1 (Figure 12).

```
$ # simulate a bad commit: break FeatureAdder, then run the CI test step
$ pytest -q -x ; echo "pytest exit code: $?"  # non-zero exit fails the GitHub Actions job
================================== FAILURES ===================================
>       assert out.loc[0, "pct_max_hr"] == np.float64(150) / (220 - 63)
E       AssertionError: assert np.float64(1.9554140127388535) == (np.float64(150.0) / (220 - 63))
E        +  where np.float64(150.0) = <class 'numpy.float64'>(150)
E        +    where <class 'numpy.float64'> = np.float64
tests/test_features.py:10: AssertionError
========================== short test summary info ============================
FAILED tests/test_features.py::test_feature_adder_values - AssertionError: as...
1 failed, 24 passed, 11 warnings in 10.34s
pytest exit code: 1
```

Figure 12 - A broken commit makes the test step fail with a non-zero exit code (real output from a local simulation).

SCREENSHOT TO ADD: Figure 13 - GitHub Actions: successful run with all jobs.

save as screenshots/10_github_actions_run.png, then re-run docs/build_report.py

SCREENSHOT TO ADD: Figure 14 - Artifacts uploaded by the run (test reports, model and tracking DB, container logs).

save as screenshots/11_github_actions_artifacts.png, then re-run docs/build_report.py

8. Containerisation

The Dockerfile uses `python:3.12-slim`, installs the slim serving requirements first (layer caching), copies only `src/` and the trained model artifacts, runs as an unprivileged user (UID 10001), declares a HEALTHCHECK and starts uvicorn on port 8000. `.dockerignore` keeps data, notebooks, tests and MLflow files out of the image.

Endpoint	Purpose
POST /predict	JSON with 13 features (ca, thal optional) → prediction, label, confidence (probability of the predicted class), probability_disease, model_version; invalid input → HTTP 422
GET /health	Liveness/readiness, model version
GET /model-info	Stored metadata: metrics, parameters, data hash
GET /metrics	Prometheus metrics

```
$ curl -s http://localhost:8000/health
{"status":"ok","model_version":"1.0.0-202610181958"}

$ curl -s -X POST http://localhost:8000/predict -d @high_risk.json
{"prediction":1,"label":"heart disease","confidence":0.989,"probability_disease":0.989,"model_version":"1.0.0-202610181958"}

$ curl -s -X POST http://localhost:8000/predict -d @low_risk_no_ca_thal.json
{"prediction":0,"label":"no heart disease","confidence":0.9756,"probability_disease":0.0244,"model_version":"1.0.0-202610181958"}

$ curl -s -w 'HTTP %{http_code}' -X POST http://localhost:8000/predict -d @invalid_age_minus3.json
{"detail":[{"type":"greater_than_equal","loc":["body","age"],"msg":"Input should be greater than or equal to 1","input":-3,"ctx":{"ge":1.0}}]}
HTTP 422
```

Figure 15 - Live API test with curl: two predictions and a rejected invalid request (real output from the local uvicorn server).

SCREENSHOT TO ADD: Figure 16 - docker build and docker run followed by the same curl tests.

save as screenshots/12_docker_build_run.png, then re-run docs/build_report.py

9. Production deployment (Kubernetes)

`k8s/` contains a Namespace, a Deployment (2 replicas, rolling update with zero unavailable, readiness and liveness probes on /health, CPU/memory requests and limits, non-root security context, Prometheus scrape annotations), a Service of type **LoadBalancer** (port 80 → 8000), an optional Ingress and a HorizontalPodAutoscaler (2-5 pods at 70% CPU), plus a kustomization file. Deploy with `kubectl apply -k k8s/`. On Minikube the image is loaded locally and `minikube tunnel` provides the external IP; on a cloud cluster the image is pulled from GHCR (published by the pipeline) after editing the `image:` line.

SCREENSHOT TO ADD: Figure 17 - kubectl get pods,svc in namespace heart-api (pods Running, service external IP).
save as screenshots/13_kubectl_get_pods_svc.png, then re-run docs/build_report.py

SCREENSHOT TO ADD: Figure 18 - Calling /health and /predict through the Kubernetes service.
save as screenshots/14_k8s_api_call.png, then re-run docs/build_report.py

10. Monitoring and logging

Logging. Every request produces one JSON log line (timestamp, request ID, method, path, status, latency, client) on stdout and in a rotating file `logs/api.log`; each prediction adds a line with the same request ID, the predicted class and confidence. Raw patient payloads are deliberately **not** logged because they are health data (a unit test enforces this).



Figure 19 - Structured request and prediction logs sharing a request_id (real output).

Metrics. Prometheus counters and histograms: `api_requests_total` (method, endpoint, status), `api_request_latency_seconds`, `predictions_total` (by class), `prediction_confidence` and `model_info`. `docker compose up` starts the API, Prometheus and Grafana with a provisioned dashboard (request rate, p95 latency, predictions by class, error count, median confidence). Prediction-class and confidence drift would be visible there; true label feedback is not available, so accuracy cannot be monitored online.

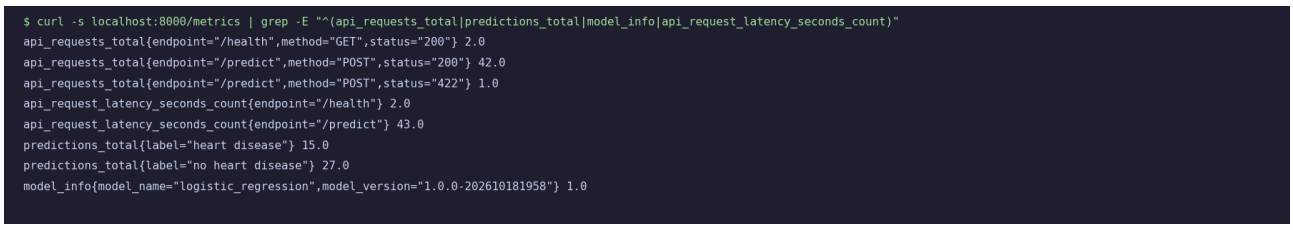


Figure 20 - /metrics output after test traffic (real output).

SCREENSHOT TO ADD: Figure 21 - Prometheus target heart-api is UP.
save as screenshots/17_prometheus_targets.png, then re-run docs/build_report.py

SCREENSHOT TO ADD: Figure 22 - Grafana dashboard under generated load.
save as screenshots/18_grafana_dashboard.png, then re-run docs/build_report.py

11. Conclusions, limitations and next steps

- A simple, interpretable model performs as well as tree ensembles on this small tabular dataset; the one-standard-error rule documents why it was chosen.
- The dataset (302 patients, single site) limits generalisation and the 61-patient test set limits precision of the estimates; the service must not be used clinically.
- Not exercised in the build environment: Docker image build, Kubernetes deployment and GitHub Actions execution (configuration was validated for syntax only; evidence screenshots are added by the author).
- Next steps: data-drift monitoring on input features, a model-registry-driven rollout, a configurable decision threshold for screening, calibration analysis and a larger multi-site dataset.

Code repository: https://github.com//heart-disease-mlops

Deployed API: not publicly hosted; run locally with Docker or Minikube as described in Sections 2, 8 and 9.